1 Overview

A flash sale puts three distributed-systems problems on one critical path: **fair admission** of traffic bigger than the system can serve, **correct inventory accounting** under contention, and **back-pressure tuning** so the purchase path never outruns the persistence layer. We built a three-service system on AWS ECS Fargate -- `waiting-room`, `flash-sale-api`, `order-worker` -- with Redis, SQS, and DynamoDB, and measured each of these problems in isolation.

Each experiment was run against a local parity stack (Docker Redis + LocalStack SQS/DynamoDB) driven by Go harnesses that mirror the load patterns we would generate against ECS. Local execution let us iterate on service code and isolate bugs that ECS would have masked as noise.

2 Experiment 1 -- Queue admission fairness

Purpose. The waiting room assigns queue positions via a Redis sorted set; two strategies were proposed for scoring: **(A) timestamp** (Unix ms) and **(B) timestamp + INCR tiebreaker**. Strategy A has a known failure mode under concurrent joins landing in the same millisecond. Experiment 1 measures how often that failure actually happens, and whether B eliminates it.

Setup. Fire N concurrent POST `/queue/join` requests with zero spawn delay against a single-replica waiting-room pointed at real Redis. Record the position each user is assigned and the request latency. Metric: **position collision rate** -- the fraction of accepted users whose assigned position is shared with at least one other user.

We used two phases: an initial local bug-discovery sweep at 100/500/1000 users, then the final validation sweep on AWS ECS/ALB at **1000 / 5000 / 10000** users for both strategies.

Bugs discovered during implementation. Running the sweep against the as-shipped code produced collision rates of 6%-26% in *both* strategies -- the B "fix" was no better than A. Diagnosis revealed two independent defects:

1. **Float64 precision loss.** Strategy B computed `score = ms + seq/1e9`. At 2026 timestamps (`ms ~ 1.77x10^12`), the float64 ULP is ~ 3.9x10^-4; the `seq/1e9` term is below precision and rounded away. The "tiebreaker" was silently a no-op.

2. **ZRANK/ZADD race.** `ZADD` and `ZRANK` ran as two separate round-trips. When a later-arriving user's member string was lexicographically smaller than an earlier one's, the later `ZADD` could push the earlier user's effective rank later than the value the earlier user had already returned, causing two users to report the same integer position at different instants.

Fix: compute the score entirely from an atomic Redis `INCR` and perform `INCR

- ZADD + ZRANK` as a single Lua script on the server. After this change, the reported numbers below are apples-to-apples: only the *scoring* differs between A and B; the atomic read-back path is identical.

Results.

\begin{center} \includegraphics[width=0.78\textwidth]{../../tests/results/charts/exp1_collision_rate.png} \end{center}

Users	Strategy A collision rate	Strategy B collision rate	Notes
1000	36.8%	**0%**	both accepted all 1000
5000	58.1%	**0%**	B accepted 4950; 50 harness timeouts
10000	66.5%	**0%**	B accepted 9612; 388 transport resets/timeouts

AWS p99 join latency at 1000 users: A = 4464 ms, B = 4367 ms. At higher loads Strategy B remained collision-free but paid extra latency because more requests survived long enough to wait on the real ALB/ECS/Redis path instead of failing fast on a duplicate-position race.

Analysis. The AWS sweep confirms the same correctness story as the local debugging phase. Strategy A degrades monotonically as concurrency rises: 36.8% collisions at 1k, 58.1% at 5k, and 66.5% at 10k. This is the exact failure mode we expected from millisecond timestamp buckets under bursty joins. Strategy B eliminates collisions completely because `INCR` guarantees a unique, strictly monotonic score and the atomic Lua path returns the rank at insert time. The small nonzero "gaps" at 5k and 10k under Strategy B were not fairness bugs: the per-user CSVs show they came from harness-side timeouts / connection resets before a `201 Created` was received.

Limitations. The final reported table is AWS-backed, but still uses a single waiting-room service behind one ALB target at a time. At 5k and 10k some requests hit client-side timeouts or transport resets before the service responded, so accepted requests are slightly below total generated load. That affects throughput and acceptance counts, but not the fairness conclusion: the accepted `timestamp_incr` joins remained collision-free across the entire sweep.

3 Experiment 2 -- Inventory correctness

Purpose. Compare three strategies for decrementing a shared inventory counter under contention: **(A) atomic DECRBY + rollback**, **(B) optimistic locking via WATCH/MULTI/EXEC**, and **(C) server-side Lua check-and-decrement**. Measure oversell and stock utilization.

Setup. Fixed inventory of 100 units; B concurrent purchase requests fire at the single-replica flash-sale-api, each requesting 1 unit. B in {200, 500, 1000}. We record the number of `201 Created` responses (accepted purchases), the final Redis counter (to detect oversell = `accepted > 100`), and throughput. Each strategy runs against a fresh process start with Redis reset between runs.

Results.

\begin{center} \includegraphics[width=0.78\textwidth]{../../tests/results/charts/exp2_accepted.png} \end{center}

Strategy	Buyers	Accepted	Oversell	Throughput (rps)
atomic	200	100	0	184

Strategy	Buyers	Accepted	Oversell	Throughput (rps)
atomic	500	100	0	456
atomic	1000	100	0	**5,102**
optimistic	200	**17**	0	1,022
optimistic	500	**12**	0	4,083
optimistic	1000	**28**	0	3,422
lua	200	100	0	135
lua	500	100	0	1,581
lua	1000	100	0	**5,821**

(First-buyer runs for each strategy include compile/JIT warmup and understate steady-state throughput; the 1,000-buyer rows are the representative numbers.)

Analysis. Atomic DECRBY and Lua both sell exactly 100 units in every configuration -- no oversell, no under-utilization. Optimistic locking is also correct (no oversell) but catastrophically wasteful: at 200 concurrent buyers it sells only 17 of the 100 available units, because every time a concurrent writer commits first the WATCH'd transaction aborts and, per the "no-retry-for-measurement" policy, we count it as a rejection. **Optimistic locking preserves correctness by sacrificing availability.** In a real flash sale this maps to "83% of the inventory never sells" -- a business failure even if the system is technically race-free.

Lua's cold-start throughput (200 buyers, 135 rps) is artificially low because the first requests pay script-compilation cost; once the script is cached, throughput matches atomic DECRBY.

Limitations. We chose not to retry aborted optimistic transactions to isolate the contention-abort rate itself. A production optimistic implementation would retry with backoff and claw back some accepted purchases, at the cost of extra round-trips. The conclusion stands: the retry budget needed to reach atomic-DECR parity under heavy contention is large enough that atomic DECR or Lua is the correct default.

4 Experiment 3 -- Admission-rate tuning and worker concurrency

Purpose. Measure how admission rate and order-worker goroutine count affect time-to-sellout, purchase latency, and SQS drain behavior under realistic flash sale load.

Setup. 1,000 concurrent users, 500 inventory units, admission rates R in {10, 50, 100}/s per replica (2 waiting-room replicas), worker goroutines in {20, 40, 80}. 9 runs on ECS Fargate, 5-minute cap per run.

Bug discovered: DynamoDB TransactionConflict. During the initial sweep we discovered that the order worker's `TransactWriteItems` -- which bundled order insert and inventory audit counter in one transaction -- caused 19% of orders to land in the Dead Letter Queue under concurrent writes. Three iterations resolved it: (v1) baseline = 94 DLQ; (v2) jitter on retry = 72; (v3) decoupled into `PutItem` + idempotent `UpdateItem` = **0**. All 9 final runs completed with zero DLQ failures.

\begin{center} \includegraphics[width=0.78\textwidth]{../../tests/results/charts/exp3_tradeoff_dual.png} \end{center}

Rate (/s)	Workers	Sell-out	/purchase p50	/purchase p99	Polls
10	20	**25.7s**	45 ms	420 ms	12,019
50	20	5.4 s	150 ms	450 ms	0
100	20	6.4 s	210 ms	820 ms	0
10	40	5.3 s	130 ms	300 ms	0
50	40	5.4 s	120 ms	450 ms	0
100	40	5.3 s	120 ms	270 ms	0
10	80	5.3 s	120 ms	240 ms	0
50	80	5.3 s	120 ms	330 ms	59
100	80	5.4 s	130 ms	330 ms	0

Zero oversell, zero DLQ failures, zero purchase errors across all 9 runs. SQS queue depth stayed at 0 throughout.

Analysis. At rate=10/s (effective 20/s), admission genuinely throttled users: sell-out took 25.7s with 12k position polls, and purchase p50 was just 45ms because load was spread over time. At rates 50 and 100, all users were admitted within the spawn window, producing ~5.3s sell-out -- but purchase contention increased sharply: p50 rose from 45ms to 210ms and p99 from 420ms to 820ms at rate=100. This is the core tradeoff: **lower admission rates produce slower sell-outs but significantly lower purchase latency.**

Worker goroutine count (20→40→80) had minimal impact on sell-out time, confirming that at 500 orders over 5 seconds, even 20 goroutines comfortably drain the SQS queue. The effect becomes visible in tail latency: at rate=100, increasing from 20 to 40 goroutines cut p99 from 820ms to 270ms, indicating that faster SQS drain reduces back-pressure on the Redis→SQS publish path.

5 Scale stress: 10,000 concurrent users

Running the experiments at 10x the planned load (10,000 concurrent joiners against one waiting-room, 10,000 concurrent buyers against one flash-sale-api) confirmed the correctness story -- zero oversell, zero position collisions under strategy B, full SQS drain -- but exposed a real operational failure that was invisible at 1,000 users.

At peak burst (~1,700 requests/sec) the waiting-room started returning `redis: connection pool timeout` on the admission ticker's background `INCR`. Root cause: go-redis's default `PoolSize` is `10 x NumCPU` (~80 on this hardware) and `PoolTimeout` is 4s. At 10k joiners the user-facing `INCR+ZADD+ZRANK` Lua starved the pool, and the ticker lost the wait race. Correctness was unaffected (the join path retried or succeeded, inventory drained cleanly to zero), but the admitted-count metric was silently undercounted, which would have been misleading in production dashboards.

A two-line fix (`PoolSize: 2000`, `PoolTimeout: 10 * time.Second` on the Redis client) eliminated the user-facing errors and all but a single cold-start tick. The lesson is the kind that only surfaces at scale: library defaults are tuned for ordinary service-to-service traffic, not for the thundering herd a flash sale creates in the first two seconds.

6 Cross-cutting conclusions

1. **Atomic operations are the right default.** Redis `INCR` (Exp 1), `DECRBY` or Lua (Exp 2), and the admission counter (Exp 3) -- every experiment confirmed that atomic primitives outperform optimistic patterns, which were either silently broken (float64 precision bug) or dramatically wasteful (83% unsold inventory).
2. **Transaction boundaries matter more than transaction guarantees.** Exp 3's DLQ bug was textbook-correct atomicity (`TransactWriteItems`) that failed at 19% under contention. Decoupling into two idempotent writes eliminated it. In high-contention paths, reducing per-write blast radius beats wrapping everything in one transaction.
3. **The admission layer shapes all downstream behavior.** When admission rate exceeded spawn rate, sell-out time and latency were determined by the load generator, not the system. The admission rate is the single most impactful operational knob -- worker goroutine count was a second-order effect at moderate scale.